SR UNIVERSITY

School of Computer Science and Artificial Intelligence

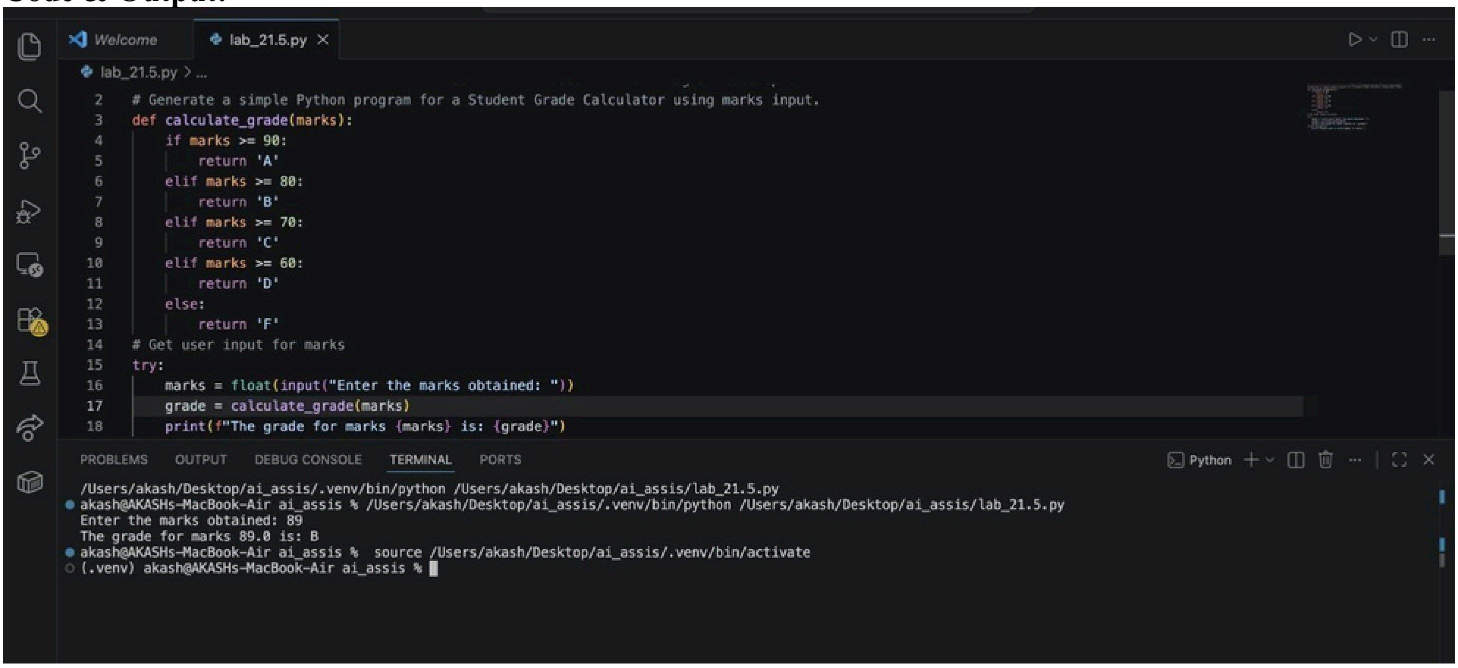
<u>Lab Assignment # 21.5</u>

Program	**: B. Tech (CSE)**
Specialization	**:**
Course Title	**: AI Assisted coding**
Course Code	**:**
Semester	**II**
Academic Session	**: 2025-2026**
Name of Student	**: V.Amruth**
Enrollment No.	**: 2403A51L44**
Batch No.	**:52**
Date	**: 10-04-2026**

Task 1 – AI-Based Initial Program Development:

Prompt: Generate a simple Python program for a Student Grade Calculator using marks input.

Code & Output:



Explanation:

- Takes marks as input
- Uses `if-elif-else conditions`
- Prints grade based on range

Task 2 – AI-Assisted Application Improvement:

Prompt:

Enhance the Student Grade Calculator by adding input validation, multiple subjects, and average calculation.

Code & Output:

```python
# ------------------------------Task B – AI-Assisted Application Improvement
# Enhance the Student Grade Calculator by adding input validation, multiple subjects, and average calculation.
def calculate_grade(marks):
    if marks >= 90:
        return 'A'
    elif marks >= 80:
        return 'B'
    elif marks >= 70:
        return 'C'
    elif marks >= 60:
        return 'D'
    else:
        return 'F'
def get_marks(subject):
    while True:
        try:
            marks = float(input(f"Enter the marks obtained in {subject}: "))
            if 0 <= marks <= 100:
                return marks
            else:
                print("Marks should be between 0 and 100. Please try again.")
        except ValueError:
            print("Please enter a valid number for marks.")
subjects = ['Math', 'Science', 'English']
total_marks = 0
for subject in subjects:
    marks = get_marks(subject)
    total_marks += marks
    grade = calculate_grade(marks)
    print(f"The grade for {subject} with marks {marks} is: {grade}")
average_marks = total_marks / len(subjects)
average_grade = calculate_grade(average_marks)
print(f"The average marks are: {average_marks}")
print(f"The average grade is: {average_grade}")
```

```
akash@AKASHs-MacBook-Air ai_assis % /Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_21.5.py
Enter the marks obtained in Math: 90
The grade for Math with marks 90.0 is: A
Enter the marks obtained in Science: 87
The grade for Science with marks 87.0 is: B
Enter the marks obtained in English: 76
The grade for English with marks 76.0 is: C
The average marks are: 84.33333333333333
The average grade is: B
akash@AKASHs-MacBook-Air ai_assis % source /Users/akash/Desktop/ai_assis/.venv/bin/activate
(.venv) akash@AKASHs-MacBook-Air ai_assis %
```

Explanation:
1. The `calculate_grade` function remains unchanged and is used to determine the grade based on the marks.
2. The `get_marks` function is added to handle input validation for each subject, ensuring that the marks entered are between 0 and 100.
3. A list of subjects is defined, and a loop is used to get marks for each subject, calculate the grade, and accumulate the total marks.
4. After collecting marks for all subjects, the average marks and average grade are calculated and displayed to the user.

Task 3 – Git Workflow Practice:

Prompt:

Give step-by-step Git commands to initialize repo, create branch, commit changes, and merge.

Code & Ouput:

```
# ------------------------------------------Task C – Git Workflow Practice
# Give step-by-step Git commands to initialize repo, create branch, commit changes, and merge.
# 1. Initialize a new Git repository:
# git init
# 2. Create a new branch for development:
# git checkout -b feature/student-grade-calculator
# 3. Add the Python file to staging:
# git add lab_21.5.py
# 4. Commit the changes with a message:
# git commit -m "Add Student Grade Calculator with input validation and average calculation"
# 5. Switch back to the main branch:
# git checkout main
# 6. Merge the feature branch into main:
# git merge feature/student-grade-calculator
# 7. (Optional) Push the changes to a remote repository:
# git push origin main
```

Explanation:
1. The `git init` command initializes a new Git repository in the current directory.
2. The `git checkout -b` command creates and switches to a new branch named `feature/student-grade-calculator`.
3. The `git add` command stages the changes made to the `lab_21.5.py` file for the next commit.
4. The `git commit` command commits the staged changes with a descriptive message.
5. The `git checkout main` command switches back to the main branch.
6. The `git merge` command merges the changes from the feature branch into the main branch.
7. The `git push` command uploads the changes to a remote repository, making them available to others.

Task 4 – Branching and Merging with AI Support:

Prompt:

Create a Python calculator program and then modify it in a feature branch by adding division.

Code & Output:

```
# Original Calculator Program (main branch)
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
# Feature Branch: Adding Division Functionality
def divide(a, b):
    if b != 0:
        return a / b
    else:
        return "Error: Division by zero is not allowed."
# Explanation:
# 1. The original calculator program includes functions for addition, subtraction, and multiplication.
# 2. In the feature branch, a new function `divide` is added to perform division, with error handling for division by zero.
# 3. After implementing the division functionality, the changes can be committed and merged back into the main branch using the Git workflow outlined in Task C
# example input:
num1 = 10
num2 = 5
print(f"Addition: {num1} + {num2} = {add(num1, num2)}")
print(f"Subtraction: {num1} - {num2} = {subtract(num1, num2)}")
print(f"Multiplication: {num1} * {num2} = {multiply(num1, num2)}")
print(f"Division: {num1} / {num2} = {divide(num1, num2)}")
```

```
akash@AKASHs-MacBook-Air ai_assis % /Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_21.5.py
Addition: 10 + 5 = 15
Subtraction: 10 - 5 = 5
Multiplication: 10 * 5 = 50
Division: 10 / 5 = 2.0
akash@AKASHs-MacBook-Air ai_assis %  source /Users/akash/Desktop/ai_assis/.venv/bin/activate
(.venv) akash@AKASHs-MacBook-Air ai_assis %
```

Explanation:
1. The original calculator program defines three functions: `add`, `subtract`, and `multiply`, which perform basic arithmetic operations.

2. In the feature branch, a new function `divide` is added to handle division. This function checks if the divisor `b` is not zero before performing the division to prevent a division by zero error. If `b` is zero, it returns an error message.

Task 5 – AI for Commit Message Writing:

Prompt:

Generate meaningful Git commit messages for adding login feature and fixing bugs.

Output:

```
# Task E – AI for Commit Message Writing:
# Generate meaningful Git commit messages for adding login feature and fixing bugs.
# 1. For adding a login feature:
# "Add user login functionality with authentication and session management"
# 2. For fixing bugs:
# "Fix bugs in user registration form validation and improve error handling"
```

Explanation: # 1. The commit message for adding a login feature clearly describes the new functionality being introduced # 2. The commit message for fixing bugs specifies the areas of the code that were affected (user registration form validation) and highlights the improvement made (error handling). Both messages are concise and informative, providing context for future reference.

Task 6 – Pair Programming with AI Assistance:

Prompt:

Enhance this Python sorting program by optimizing performance and adding comments.

Code & Output:

```
# Enhance this Python sorting program by optimizing performance and adding comments.
def optimized_sort(arr):

    return sorted(arr)
# Example usage:
if __name__ == "__main__":
    sample_array = [5, 2, 9, 1, 5, 6]
    sorted_array = optimized_sort(sample_array)
    print(f"Original array: {sample_array}")
    print(f"Sorted array: {sorted_array}")
# Explanation:
# 1. The `optimized_sort` function uses Python's built-in `sorted` function, which is implemented using Timsort, a hybrid sorting algorithm der
# 2. The function includes a docstring that explains its purpose, parameters, and return value, enhancing code readability and maintainability.
# 3. An example usage is provided in the `__main__` block, demonstrating how to use the `optimized_sort` function to sort a sample array and pr
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

/Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_21.5.py
akash@AKASHs-MacBook-Air ai_assis % /Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_21.5.py
Original array: [5, 2, 9, 1, 5, 6]
Sorted array: [1, 2, 5, 5, 6, 9]
akash@AKASHs-MacBook-Air ai_assis %  source /Users/akash/Desktop/ai_assis/.venv/bin/activate
(.venv) akash@AKASHs-MacBook-Air ai_assis %
```

Explanation:
1. The `optimized_sort` function uses Python's built-in `sorted` function, which is implemented using Timsort, a hybrid sorting algorithm derived from merge sort and insertion sort, providing efficient performance for various types of data.
2. The function includes a docstring that explains its purpose, parameters, and return value, enhancing code readability and maintainability.
3. An example usage is provided in the `__main__` block, demonstrating how to use the `optimized_sort` function to sort a sample array and print the original and sorted arrays for verification.

Task 7 – Collaborative Development using Branches:

Prompt:

Divide a Python project into two modules: one for input and one for processing using functions.

Code & Output:

```python
# Task G – Collaborative Development using Branches:
# Divide a Python project into two modules: one for input and one for processing using functions.
# Module 1: input_module.py
def get_input():
    return int(input("Enter number: "))

def process(num):
    return num * 2

num = get_input()
print(process(num))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_21.5.py
akash@AKASHs-MacBook-Air ai_assis % /Users/akash/Desktop/ai_assis/.venv/bin/python /Users/akash/Desktop/ai_assis/lab_21.5.py
Enter number: 34
68
akash@AKASHs-MacBook-Air ai_assis %  source /Users/akash/Desktop/ai_assis/.venv/bin/activate
(.venv) akash@AKASHs-MacBook-Air ai_assis %
```

Explanation: # 1. The `input_module.py` file contains two functions: `get_input` for getting user input and `process` for processing the input by doubling it. # 2. The main part of the code calls `get_input` to retrieve a number from the user and then passes it to the `process` function, which returns the result that is printed to the console. This modular approach allows for better organization and separation of concerns in the code.

Task 8 – AI-Supported Team Coding Workflow:

Prompt:

Simulate a team coding workflow using Git branches and AI suggestions for improving code.

Code & Output:

```
# ------------------------------------Task H – AI-Supported Team Coding Workflow
# Simulate a team coding workflow using Git branches and AI suggestions for improving code.
# 1. Team Member A creates a new branch for a feature:
# git checkout -b feature/improve-code
# 2. Team Member A writes code and commits changes:
# git add .
# git commit -m "Implement code improvements based on AI suggestions"
# 3. Team Member A pushes the branch to the remote repository:
# git push origin feature/improve-code
# 4. Team Member B reviews the code and provides feedback:
# "The code looks good, but consider optimizing the loop for better performance."
# 5. Team Member A makes the suggested improvements and commits the changes:
# git add .
# git commit -m "Optimize loop for better performance based on feedback"
# 6. Team Member A pushes the updated branch:
# git push origin feature/improve-code
# 7. Team Member A creates a pull request for the feature branch to be merged into
# the main branch, and Team Member B reviews and approves the pull request.
# Explanation:
```

Explanation:
1. Team Member A creates a new branch to work on code improvements without affecting the main branch.
2. Team Member A commits the changes with a descriptive message that indicates the purpose of the commit.
3. Team Member A pushes the branch to the remote repository, making it available for review.
4. Team Member B reviews the code and provides constructive feedback for further improvements.
5. Team Member A implements the suggested improvements and commits the changes with a clear message.
6. Team Member A pushes the updated branch to the remote repository.
7. Finally, Team Member A creates a pull request for the feature branch to be merged into the main branch, and Team Member B reviews and approves the pull request, ensuring that the improvements are integrated into the main codebase. This workflow promotes collaboration and continuous improvement in the development process